

operator

Base class for all Plan Operators.

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

BaseFeaturesOperator

```
BaseFeaturesOperator(pyjnius_java)
```

Bases: [BaseOperator](#) , ABC

Abstract base class to provide additional Python functionality to Plan Operators containing features.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_feature

```
add_feature(feature: BaseFeature, logger: bool = True) -> BaseFeature
```

Adds a new Feature to this Operator.

Parameters:

Name	Type	Description	Default
feature 	BaseFeature	New Feature.	<i>required</i>
logger 	bool	Whether to log info message or not.	True

Returns:

Type	Description
BaseFeature	New Feature.

diff

```
diff(other: BaseFeaturesOperator, only_diff: bool = True)
```

Display the differences between the features from this Operator and another Operator.

Features are compared based on their feature name. The order of the features in the Operator is not taken into account.

Parameters:

Name	Type	Description	Default
other Â¶	<code>BaseFeaturesOperator</code>	BaseFeaturesOperator to compare against.	<i>required</i>
only_diff Â¶	<code>bool</code>	Show features with differences only. Defaults to 'true'.	<code>True</code>

Raises:

Type	Description
<code>NotImplementedError</code>	When not called from a Jupyter notebook.

get_dependencies [Â¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

get_dependencies_by_type [Â¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, list[str]]]
```

get_feature `abstractmethod` [Â¶](#)

```
get_feature(name: str)
```

get_features `abstractmethod` [Â¶](#)

```
get_features()
```

get_type `abstractmethod` `staticmethod` [Â¶](#)

```
get_type() -> str
```

Gets the operator type.

Returns:

Type	Description
<code>str</code>	Operator type.

remove_feature [Â¶](#)

```
remove_feature(name: str) -> bool
```

Removes the features that match the name.

Parameters:

Name	Type	Description	Default
name Â¶	<code>str</code>	Name of the feature to remove.	<i>required</i>

Returns:

Type	Description
<code>bool</code>	

Type	Description
	If the feature is removed.

rename_feature [↗](#)

```
rename_feature(old_name: str, new_name: str)
```

to_csv [↗](#)

```
to_csv(path: Union[Path, str])
```

Exports all the features in the operator as a csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	Path or str	Path to where the csv will be written.	<i>required</i>

to_dict [↗](#)

```
to_dict() -> List[Dict]
```

to_pandas [↗](#)

```
to_pandas() -> DataFrame
```

Exports all the features in the operator as a pandas dataframe.

Returns:

Type	Description
DataFrame	Pandas dataframe containing features and all their editable attributes.

update_from_csv [↗](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Operator with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	Path or str	Path from where the csv will be read.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

update_from_dict [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```

update_from_pandas [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Operator with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>pd_df</code> ¶	<code>DataFrame</code>	Pandas dataframe with the operator features.	<i>required</i>

See Also

`Plan.from_csv` : For guidelines to input csv data in the right format.

BaseOperator [¶](#)

```
BaseOperator(pyjnius_java)
```

Bases: `JavaWrapper` , `ABC`

Abstract class to provide additional Python functionality to all Plan Operators.

Notes

When comparing operators we consider two operators to be the same if they are of the same type and have the same name. This allows a more intuitive comparison between plans where we want to analyze differences of "the same" operator. In such cases we need to first identify the "same" operator between plans so that then we can check how their contents are different.

For reference, `BaseOperator.getName` returns the name of the operator as it appears in the UI.

subclasses **class-attribute** **instance-attribute** [¶](#)

```
subclasses: List[JavaWrapper] = []
```

get_dependencies **abstractmethod** [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

Gets the features and their direct dependencies within the operator.

Dependencies are not recursive.

Returns:

Type	Description
<code>dict</code>	Dictionary of FeatureUID and the name of their direct dependencies.

get_dependencies_by_type **abstractmethod** [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, List[str]]]
```

Gets the features and their direct dependencies (by type) within the operator.

Dependencies are not recursive.

Returns:

Type	Description
<code>dict</code>	Dictionary of FeatureUID and the dict mapping the types and names of their direct dependencies.

`get_type` abstractmethod staticmethod [¶](#)

```
get_type() -> str
```

Gets the operator type.

Returns:

Type	Description
<code>str</code>	Operator type.

`rename_feature` abstractmethod [¶](#)

```
rename_feature(old_name: str, new_name: str)
```

Renames the dependencies or features matching the old name.

Parameters:

Name	Type	Description	Default
<code>old_name</code> ¶	<code>str</code>	Old name to change from.	<i>required</i>
<code>new_name</code> ¶	<code>str</code>	New name to change to.	<i>required</i>

`to_dict` abstractmethod [¶](#)

```
to_dict() -> List[Dict]
```

Exports all the features in the operator as a list of dictionaries.

Returns:

Type	Description
<code>list of dict</code>	List containing features and all their editable attributes.

`update_from_dict` abstractmethod [¶](#)

```
update_from_dict(dict_feats: List[Dict])
```

Replaces the operator features with the ones on the list.

Parameters:

Name	Type	Description	Default
<code>dict_feats</code> ¶	<code>list of dict</code>	List containing the features in a dict format.	<i>required</i>